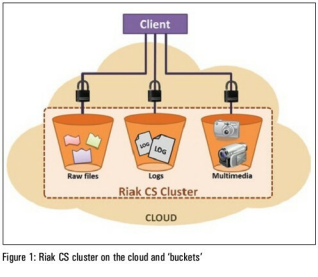

Figure 1: Riak CS cluster on the cloud and 'buckets'

developed by Basho that allows organisations to build scalable, highly available storage services. Built on top of a highly scalable no-SQL database called Riak, Riak CS offers multi-tenant storage capabilities for large objects as well as compatibility for Amazon S3 (Simple Storage as a Service). It can be leveraged for building public or private clouds, or even as reliable storage for applications and platforms.

A few important features that Riak CS provides are listed below.

Large object support: Riak CS is designed and built to store very large objects. Each object gets stored in a container called a 'bucket', which can handle objects of any type, ranging from multi-media content such as videos, images, text files to application logs, database backups, software files, etc (Figure 1). Users can leverage a unique feature provided by Riak CS called a Multi-Part Upload to upload large objects—even those that range in terabytes. This feature breaks the large object into smaller, manageable chunks and streams them into the underlying Riak cluster. Here, the object is stored and replicated, thus making it highly available and fault tolerant.

The Amazon S3 compatible API: Riak CS enables organisations to create an internal Amazon Web Services S3-compatible storage service that can provide cost savings as well as better integration capabilities with Amazon's own public cloud. This proves to be extremely useful in case an organisation wants to scale out its internal storage capacity in a very short span of time using a hybrid cloud storage approach, without having to make any changes in the underlying APIs.

Access control and authentication: Riak CS provides access control lists (ACLs) as a means to grant and deny access to objects and buckets alike. Each bucket and object is associated with an ACL that can be used to provide permissions such as read/write and full control.

Simple operational model for scaling: Scaling a Riak CS cluster is a simple and very straightforward process. You simply have to add additional nodes to the cluster, and Riak CS will automatically redistribute data across the newly added machines. The same applies even if you remove nodes from a Riak cluster. This eliminates management complexities and allows the cloud administrators to scale their storage in either a public or a private cloud environment with ease.

Integration with most open source cloud platforms: Riak CS can be easily integrated with existing private cloud platforms such as CloudStack, OpenStack and Eucalyptus to provide a distributed and scalable storage for your cloud users.

Multi-datacentre replication for active backups, disaster recovery and data locality: Riak offers a commercial version in the form of Riak CS Enterprise, which provides added features such as multi-datacentre replication and disaster recovery. It leverages either real-time or full sync capabilities to replicate data across multiple datacentres, thus providing active availability zones that provide data storage redundancy.

Riak CS architecture

Before we actually get into working with Riak CS, it is essential that we first understand how Riak CS is actually designed. As mentioned earlier, Riak CS is built on top of Basho's highly available, distributed no-SQL database called Riak.

Riak internally manages distribution of data when new nodes are either added or removed from a Riak cluster. It even manages to replicate the data (three times, by default) over the cluster, so that your data is always in a state of availability in spite of some random node failure. All this is made possible by a consistent hashing algorithm that Riak uses internally and, hence, so does Riak CS.

In a Riak CS system, all underlying nodes have Riak installed and configured in them. When a large object is uploaded to Riak CS by using a client (as shown in Figure 2), it first gets broken down into smaller, more manageable pieces called 'chunks'. Each chunk is then written and replicated inside the Riak cluster and associated with unique metadata as well. The client can use this metadata to retrieve the object, as and when required.

One important aspect of the Riak CS architecture is that there is no concept of a 'master-slave' node. Here, all nodes share the same responsibility and replicate the data internally.

Here's an explanation of the Riak CS architecture shown in Figure 2.

Step 1: The large object is first uploaded to the Riak CS node via a client.

Step 2: Riak CS breaks down the large object into smaller, more manageable chunks (1MB, by default).

Step 3: These chunks are then passed down to the corresponding Riak node, which in turn, takes care of data replication as well as metadata creation.